QCMS

KNOWLEDGE TRANSFER DOCUMENT

Quality & Continuous Improvement Management System

QCMS Enterprise OS

Complete Software KT Documentation — Architecture, Installation, Configuration & Operations Guide

VERSION	DATE	CLASSIFICATION	STATUS
1.2.1	August 2026	Internal Engineering	Active / Production

Table of Contents

SECTION 1

System Overview & Purpose

QCMS (Quality & Continuous Improvement Management System) is an enterprise-grade, multi-tenant SaaS platform that digitizes and automates structured problem-solving methodologies for industrial organizations — primarily **Quality Circle (QC Story / 8D)**, **Six Sigma / DMAIC**, and continuous improvement programs.

What Problem Does It Solve?

Most manufacturing and service organizations track quality improvement projects on paper, Excel sheets, or disconnected tools. QCMS provides a single digital platform where the entire lifecycle — from problem identification to SOP standardization — is tracked, governed, and measurable.

Who Uses It?

- Quality Circles / Kaizen Teams
- Manufacturing Plants & Factories
- ISO 9001 Compliance Teams
- Six Sigma Black/Green Belt Programs
- Corporate Quality Departments

Core Capabilities

🔄 8-Stage Workflow

Rigid sequential problem-solving lifecycle with approval gates and role-locked stage transitions.

🏢 Multi-Tenant SaaS

Complete data isolation per organization. Shared DB, shared schema, row-level org_id scoping.

🤖 Quality AI (RAG)

AI-powered semantic search across closed projects using vector embeddings and cosine similarity.

📊 Enterprise Analytics

KPI dashboards, trend analysis, savings verification, and CEO-level financial reporting.

📄 Automated PDF Reports

Auto-generated QC Story closures, ISO certificates, invoices, and compliance audit reports.

🌐 Multi-Language i18n

Real-time UI translation in 6 languages: English, Hindi, Kannada, Telugu, Tamil, Malayalam.

SECTION 2

Technology Stack

Layer	Technology	Version / Detail	Purpose
Frontend UI	HTML5, Vanilla JavaScript (ES6+)	No Framework	Glassmorphism design system, SPCI pattern
UI Styling	Vanilla CSS3	CSS Variables + Glass.css	Design tokens, glassmorphism aesthetic layer
Charts & Icons	Chart.js, Lucide Icons	Latest CDN	KPI visualization, iconography
Backend API	Python + Flask	Python 3.11, Flask 3.0.0	REST API, Blueprint-based modular routing
ORM	SQLAlchemy	2.0.23	Type-safe DB modeling, query builder
Auth	Flask-JWT-Extended + Bcrypt	4.5.3 / 1.0.1	Stateless JWT auth, password hashing
Database	PostgreSQL	Neon Serverless / Local 15+	Primary relational store + JSONB fields
AI / Embeddings	all-MiniLM-L6-v2 + FAISS	384-dim vectors	RAG semantic search over closed projects

Layer	Technology	Version / Detail	Purpose
Email	Resend API	resend 2.4.0	Transactional emails, OTP, notifications
PDF Engine	fpdf2	2.7.5	QC Story PDFs, invoices, certificates
Payments	Razorpay	Webhook-based	Subscription upgrades, billing invoices
Migration	Alembic + Flask-Migrate	1.13.1	DB schema versioning
WSGI Server	Gunicorn	21.2.0	Production app server, single worker, preload
Reverse Proxy	Nginx (Alpine)	Latest	Static file serving, API proxy, SSL termination
Containerization	Docker + Docker Compose	Latest	Consistent environment, multi-service orchestration

SECTION 3

Architecture — Clean Architecture Layers

QCMS backend follows strict **Clean Architecture** (Domain → Application → Infrastructure → Presentation), separating business rules from I/O concerns. This makes the system testable, replaceable, and independently deployable.

Layer	Location	Responsibility	Key Files
Presentation	`app/presentation/`	HTTP routing, request validation, JWT guards, response serialization	35 Blueprint route files, middleware/
Application	`app/application/`	Use-case handlers, orchestrates domain + infrastructure calls	Service handlers per feature domain
Domain	`app/domain/`	Pure business logic, policy engines, feature flags, exceptions	services/, exceptions/
Infrastructure	`app/infrastructure/`	SQLAlchemy ORM models, DB sessions, file I/O, email adapters	database/models/models.py
Config	`app/config/settings.py`	All environment variable loading, DB URL parsing, CORS, pool settings	settings.py
Utils	`app/utils/`	PDF template fillers, report generators, avatar helpers	pdf_template_helper.py (47KB)

App Factory Pattern

The Flask app is created via `create_app()` in `backend/app/__init__.py`. All 35 blueprints, security middleware, JWT hooks, CORS, and DB auto-migration are initialized here. Gunicorn calls this with `preload_app=True` so migrations only run once in the master process before worker forking.

SECTION 4

Directory Structure

```
/ (Repository Root) ├── backend/ # Flask REST API Service | ├── app/ | | ├── __init__.py # App factory, blueprint registration, JWT hooks | | ├── config/ | | | └── settings.py # All env vars, DB URL parsing, pool config | | ├── domain/ | | | ├── services/ # Business logic, subscription manager, policies | | | └── exceptions/ # Custom exception classes | | ├── application/ # Use-case handlers, feature orchestration | | ├── infrastructure/ | | | └── database/models/ | | | └── models.py # ALL SQLAlchemy ORM models (single file) | | ├── presentation/ | | | ├── routes/ # 35 Blueprint files (API endpoints) | | | └── middleware/ # Security, JWT, idempotency, WAF | | └── utils/ # PDF helpers, report builders | ├── run.py # WSGI entry point (Gunicorn: run:app) | ├── gunicorn.conf.py # Gunicorn: 1 worker, 300s timeout, preload | ├── requirements.txt # Python dependencies | ├── .env # Secrets (NOT committed to git) | └── Dockerfile # python:3.11-slim, gunicorn CMD | ├── frontend/ # Static Web Application | ├── index.html # Public landing page | ├── assets/ | | ├── css/ # design-system.css, glass.css, utilities | | ├── js/ # 30+ JS modules (components.js 178KB, super-admin.js 643KB) | | └── translations/ # i18n JSON dictionaries (6 languages) | ├── admin/ # Super Admin portal HTML pages | ├── auth/ # Login, register, password recovery | ├── dashboard/ # Role dashboards (Admin, CEO, Reviewer, TL) | ├── projects/ # 8-Stage workspace, QC tools, repository | ├── analytics/ # KPI dashboards, financial reports | ├── nginx.conf # Reverse proxy: /api → backend:5000 | └── Dockerfile # nginx:alpine serving static files | ├── migrations/ # Alembic DB migration scripts ├── docker-compose.yml # Backend:5000 + Frontend:80 orchestration └── .env.local # Root-level environment overrides
```

SECTION 5

Role-Based Access Control (RBAC)

QCMS enforces a strict **6-tier hierarchical permission model**. Every API endpoint is protected by a `role_required` decorator that compares the authenticated user's role level against the endpoint's minimum required level.

Level	Role	Scope	Key Permissions
Platform	Super Admin	Entire Platform	Manage ALL organizations, subscriptions, billing, system settings, global announcements, integrations, platform identity/branding
Level 4	Admin	Own Organization	Create/manage users, departments, plants. Configure org settings, security policies, workflow templates. View all org analytics.
Level 3	Reviewer	Own Organization	Quality gatekeeper. Mandatory sign-off on Stages 4, 5, 7, 8. Can reject with comments. Cannot edit project data.
Level 2	Facilitator	Own Organization	Technical SME. Validates Root Cause Analysis (Stage 5). Can oversee multiple teams. Access to QC tools and cross-project data.
Level 1	Team Leader	Own Projects	Project manager. Creates and drives projects. Assigns tasks to Team Members. Responsible for stage completion.
Level 0	Team Member	Assigned Projects	Data entry and task execution. Can only work on projects they are explicitly assigned to.

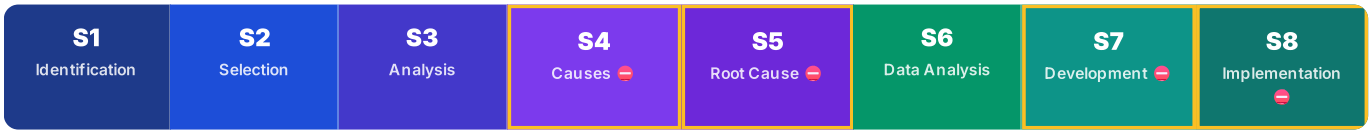
How the Guard Works in Code

Each API route uses `@jwt_required()` + a custom `@role_required(min_level)` decorator. If the user's level is below the required level, the API returns HTTP 403. Super Admin tokens carry an out-of-band `is_super_admin=True` claim and bypass org_id scoping entirely.

SECTION 6

The 8-Stage Quality Circle Workflow Engine

The core of QCMS is a **rigid, sequential 8-stage lifecycle**. A project cannot advance to the next stage without completing mandatory fields and, for gated stages, obtaining explicit Reviewer/Facilitator approval.



⛔ = Gated stage requiring explicit Reviewer approval before transition

Stage	Name	Key Activities	Approval Required	DB Model
1	Identification	Problem statement, team formation, target metrics, project charter	No	Stage1Identification
2	Selection	KPI baseline data collection, data stratification, evidence upload	No	Stage2Selection
3	Analysis	Fishbone diagram, 5-Why tree, Pareto analysis, hypothesis testing	No	Stage3Analysis
4	Causes	Probable cause identification, verification of root cause candidates	**Reviewer ✔**	Stage4Causes

Stage	Name	Key Activities	Approval Required	DB Model
5	Root Cause	Final RCA sign-off, statistical validation, Facilitator verification	**Facilitator + Reviewer ✔**	Stage5RootCause
6	Data Analysis	Countermeasure planning, cost-benefit analysis, ROI estimation	No	Stage6DataAnalysis
7	Development	Implementation milestones, task assignments, execution tracking	**Reviewer ✔**	Stage7Development
8	Implementation	Post-KPI measurement, savings verification, SOP update, lessons learned	**Reviewer ✔**	Stage8Implementation

QC Tools Available at Stage 3

Interactive **Fishbone Diagram** (6M categories: Man, Machine, Method, Material, Measurement, Environment), **5-Why Analysis Tree**, **Pareto Chart** builder, **Histogram**, **Control Chart** — all rendered client-side via Canvas/SVG and stored as JSON in the DB.

SECTION 7

API Modules — All 35 Blueprint Route Files

Every feature area is a separate Flask Blueprint registered with its own URL prefix. This ensures clean separation of concerns and allows independent development of each module.

Blueprint File	URL Prefix	Size	Key Endpoints / Responsibility
auth_routes.py	/api/auth	78KB	Login, logout, registration, OTP verify, password reset, Google SSO, magic links, user profile update
project_routes.py	/api/projects	95KB	Create/read/update/delete projects, assign teams, project storyboard, status transitions
workflow_routes.py	/api/workflow	22KB	Stage progression engine, approval/rejection logic, stage validation
admin_routes.py	/api/admin	164KB	User management, department CRUD, org settings, custom fields, security policies, audit logs, rewards
plant_routes.py	/api/admin/plants	14KB	Multi-plant management: create plants, assign users/departments to plants
analytics_routes.py	/api/analytics	129KB	KPI dashboards, trend analytics, delivery rates, department performance, CEO-level reports
super_admin_routes.py	/api/super-admin	178KB	Multi-tenant org management, subscription control, platform settings, global branding, security
super_admin_v1_routes.py	/api/v1	63KB	V1 API compatibility layer for super admin operations
ceo_routes.py	/api/ceo	71KB	Executive-level financial analytics, cross-department KPIs, enterprise savings reports
reviewer_routes.py	/api/reviewer	51KB	Pending review queue, approve/reject stage gates, review history, reviewer dashboard data
facilitator_routes.py	/api/facilitator	33KB	RCA validation, facilitation oversight, team progress monitoring
team_leader_routes.py	/api/team-leader	17KB	Team assignment, task management, project progress for TL dashboard
team_member_routes.py	/api/team-member	4KB	Assigned project data access, task completion endpoints
qc_tools_routes.py	/api/project	41KB	Fishbone diagram, 5-Why, Pareto, Histogram, Control Chart data save/load
dashboard_routes.py	/api/dashboard	10KB	Role-tailored dashboard summary cards, recent activity, KPI widgets
repository_routes.py	/api/repository	39KB	Knowledge repository CRUD, closed project indexing, embedding ingestion triggers
rag_routes.py	/api/rag	5KB	Quality AI chat endpoint — receives query, runs cosine similarity, returns ranked past solutions
subscription_routes.py	/api/subscriptions	135KB	Plan management, trial extensions, upgrade/downgrade, Razorpay webhook handling
billing_routes.py	/api/billing	60KB	Invoice generation, payment history, billing address, GST invoice PDFs
license_routes.py	/api/licenses	23KB	License key validation, expiry management, multi-seat enforcement

Blueprint File	URL Prefix	Size	Key Endpoints / Responsibility
modules_routes.py	/api/modules	31KB	Feature module enable/disable per org, subscription-gated feature flags
reports_routes.py	/api/reports	36KB	PDF report generation: QC closure reports, compliance summaries, stage-by-stage exports
sop_routes.py	/api/sops	87KB	Standard Operating Procedure CRUD, version control, SOP linking to projects
audit_routes.py	(root /api)	70KB	Audit log queries, activity stream, risk-level events, user session tracking
support_routes.py	/api/support	51KB	Support ticket creation, status tracking, admin response, SLA monitoring
announcement_routes.py	(root /api)	57KB	Global/org-level announcement broadcast, read receipts, audience targeting
notification_routes.py	/api	5.5KB	In-app notification delivery, mark-as-read, notification preferences
email_notification_routes.py	(root /api)	44KB	Transactional email triggers via Resend API, email template management
integrations_routes.py	/api/super-admin	28KB	Third-party integration management (Razorpay, webhooks)
integration_v1_routes.py	/api/v1/integrations	19KB	V1 integration API for external system connectivity
points_routes.py	/api	19KB	Gamification points system, leaderboards, badges, rewards redemption
document_branding_routes.py	(root /api)	31KB	Dynamic platform branding: name, logo, colors, acronym, invoice headers
feature_engine_routes.py	(root /api)	5KB	Feature flag evaluation for frontend conditional rendering
workflow_routes.py	/api/workflow	22KB	Stage transition rules, approval workflow configuration
dashboard_activity_route.py	/api/dashboard	1.4KB	Live activity feed for dashboard home widget

SECTION 8

Frontend Architecture — SPCI Pattern

The frontend uses **no framework** (no React/Vue/Angular). Instead, it uses a custom architectural pattern called **SPCI (Single Page Component Injection)** where UI components are dynamically generated and injected into the DOM via JavaScript functions.

Core JavaScript Modules

File	Size	Purpose
`components.js`	178KB	CORE UI ENGINE: renderSidebar, renderNavbar, kpiCard, statusBadge, modal builders — adapts to all 6 roles
`super-admin.js`	643KB	Super Admin portal — full org management, subscription tables, platform settings panels
`platform-settings.js`	153KB	Org-level settings: branding, security, workflow config, notification templates
`project-details-app.js`	156KB	8-Stage workspace: all stage renderers, QC tools, review panels, PDF export
`support-desk.js`	163KB	Full support ticket system UI
`announcements.js`	332KB	Global announcement system with rich text editor
`auth-guard.js`	65KB	Frontend RBAC enforcement, route protection, permission utilities
`api.js`	25KB	Service layer: auto-injects org_id and JWT headers, handles 401/403 centrally
`i18n.js`	27KB	Real-time i18n engine with DOM MutationObserver for dynamic content
`enterprise-analytics.js`	63KB	Chart.js-based KPI dashboards, trend analysis

CSS Design System

Glassmorphism Design

The UI uses a custom Glassmorphism design system with:

- `design-system.css` — CSS variables for spacing, colors, typography tokens
- `glass.css` — `backdrop-filter: blur()` aesthetic layer
- Dark/Light mode via `data-theme` attribute + `ThemeManager`

Frontend Page Structure

- `index.html` — Public landing page / marketing site
- `auth/` — Login, register, password reset, OTP verify
- `dashboard/` — Role-specific dashboards
- `projects/` — Project list, detail workspace, repository
- `analytics/` — KPI analytics, savings reports
- `admin/` — Super Admin portal pages
- `docs/` — Internal documentation pages

Multi-Language Support

JSON dictionaries in `assets/translations/` for: **en, hi, kn, te, ta, ml**. The `i18n.js` module watches DOM mutations and auto-translates dynamically injected content.

SECTION 9

Database Schema Overview

All models are defined in a single file: `backend/app/infrastructure/database/models/models.py`. The database uses **PostgreSQL** with **JSONB** columns for flexible fields.

Core Tables

Table	Key Columns / Notes
organizations	org_id, subscription_plan, org_scale, plants, security_settings (JSONB), login_options (JSONB), enabled_modules (JSONB)
users	user_id, org_id, role, email, is_active, plant_id, custom_fields (JSONB), deactivated_at
departments	id, org_id, plant_id, name
plants	id, org_id, name, code, location
projects	id, org_id, title, status, team_id, current_stage
project_workflow	id, project_id, stage_data (JSONB), template_snapshot (JSONB)
stage1_identification	project_id, problem_statement, target_kpi, team_members
stage3_analysis	project_id, fishbone_data (JSONB), five_why_data (JSONB)
knowledge_repository	id, org_id, title, problem_summary, root_cause, solution, embedding (vector)

SaaS / Billing Tables

Table	Key Columns / Notes
saas_plans	id, name, max_users, max_projects, is_default_trial, trial_duration_days
subscriptions	id, org_id, plan_id, status, trial_start/end_date, cancelled_at
saas_user_sessions	id, user_id, session_id, status (Active/Terminated/Revoked)
audit_logs	id, org_id, user_id, action, resource, risk_level, timestamp
platform_settings	id, branding_config (JSONB), global_template_version
_qcms_schema_version	version (INT) — migration version guard

Schema Auto-Migration

On first boot, 100+ ALTER TABLE statements run automatically. On subsequent boots, the `_qcms_schema_version` table is checked — if the version matches, all migrations are skipped for fast startup (<2s).

SECTION 10

Security & Middleware Stack

Middleware Layers (backend)

Middleware	Purpose
JWT Auth Guard	Validates Bearer token on every /api/* request. Checks revocation via SaaSUserSession table.
Role Required	Enforces RBAC hierarchy (0-4). Returns 403 if user level below endpoint minimum.
Subscription Guard	Feature-flags endpoints by org subscription tier (Starter/Professional/Enterprise).
WAF (Web App Firewall)	SQL injection, XSS pattern detection on request bodies. DB-driven — enabled via security_settings.
Rate Limiter	Per-IP request throttling. Brute-force lockout on /api/auth/login after N failures.
IP Whitelist/Blacklist	org-level IP restriction policy stored in security_settings JSONB.
Idempotency	Deduplicates concurrent identical POST requests using Idempotency-Key header.
Security Headers	X-Content-Type-Options, X-Frame-Options, strict CORS on /api/*.

JWT Session Lifecycle

1. User logs in → JWT issued with `session_id` claim + SaaSUserSession row created (status=Active)
2. Every subsequent request → `token_in_blocklist_loader` checks if session is Terminated/Revoked
3. Admin can force-terminate any active session → SaaSUserSession row updated → next request instantly returns 401
4. Deactivated/deleted users → `is_active=False` check revokes all tokens immediately

Security Settings per Org (JSONB)

- Enable/disable WAF, rate limiting, IP whitelist
- Set login attempt limits, lockout duration
- Force 2FA / OTP on login
- Custom session timeout values
- Allowed IP ranges for admin access

SECTION 11

Quality AI — RAG Intelligent Search

The **Quality AI** feature allows engineers to ask natural language questions like *"How was paint peeling on bumpers solved before?"* and retrieve verified past solutions from closed projects — without needing to match exact keywords.

How It Works (RAG Pipeline)

1. **Ingestion:** When a project is closed and approved, its problem summary, root cause, and solution are extracted and converted to a 384-dimensional embedding vector using `all-MiniLM-L6-v2` neural network.
2. **Storage:** The vector is stored in the `knowledge_repository` table alongside the full project metadata.
3. **Query:** User types a question in the AI chat widget → query is converted to a 384-dim vector.
4. **Retrieval:** Cosine similarity computed between query vector and all stored project vectors (locked to org_id).
5. **Ranked Answer:** Top 3-4 most similar past projects returned with root cause, countermeasure, KPI %, and cost savings.

API Endpoint

```
POST /api/rag/chat Auth: Bearer JWT required
Org: Locked to org_id from token Body: {
"query": "How was paint peeling solved?" }
Response: { "answer": "Top 2 matching
solutions ...", "sources": [ { "project_id": 84,
"similarity": 0.97 }, { "project_id": 19,
"similarity": 0.91 } ] }
```

Subscription Gate

Quality AI is available only on **Enterprise tier** subscriptions. The `subscription_guard` middleware blocks access for Starter/Professional plans with HTTP 403.

SECTION 12

Subscription & Multi-Tenant Model

Starter

- 5 Active Projects
- 10 Users
- Basic Analytics
- No AI / No API

Professional

- 50 Projects
- 100 Users
- Advanced Analytics
- White-labeling
- Priority Email

Enterprise

- Unlimited everything
- Quality AI (RAG)
- API Access
- Multi-plant support
- Priority Support SLA

Trial extensions: Up to 2 auto-approved. The `max_auto_trial_extensions` is set in `platform_settings`. Razorpay integration handles payment → webhook → subscription upgrade.

SECTION 13

Installation Guide — Docker (Recommended for Production)

Prerequisites for Docker Installation

Docker 24+ and Docker Compose v2+ must be installed on the host. No Python or Node.js needed on the host when using Docker.

Step 1 — Clone the Repository

```
# SSH (recommended for servers) git clone git@github.com:harshith1432/imfq-QCMS.git cd imfq-QCMS # OR via HTTPS git clone https://github.com/harshith1432/imfq-QCMS.git cd imfq-QCMS
```

Step 2 — Create the Backend Environment File

```
# Copy the example and fill in your actual values cp backend/.env.example backend/.env # Open backend/.env and set: DATABASE_URL=postgresql://user:password@host:5432/dbname JWT_SECRET_KEY=your_strong_random_secret_here RESEND_API_KEY=re_xxxxxxxxxxxx RESEND_FROM_EMAIL=noreply@yourdomain.com SUPER_ADMIN_USERNAME=admin@yourcompany.com SUPER_ADMIN_PASSWORD=StrongAdminPassword123! INTEGRATION_BASE_URL=https://yourdomain.com CORS_ORIGINS=https://yourdomain.com PORT=10000
```

Step 3 — Build and Start All Services

```
# Build images and start containers in background docker-compose up --build -d # Verify both containers are running docker-compose ps # Watch startup logs (migration runs on first boot) docker-compose logs -f backend
```

Step 4 — Verify the Deployment

```
# Frontend web app (served by Nginx) curl http://localhost:80 # Backend API health curl http://localhost:5000/api/auth/health # Expected: {"status": "ok"} # On first boot you will see in logs: # [QCMS] Connecting to database at: postgresql://user:****@host:5432/dbname # [QCMS] Running schema migrations... # [QCMS] Schema migrations complete. Version: 2
```

First-Boot Migration Time

On the very first deployment against a fresh database, 100+ ALTER TABLE statements run automatically. This takes approximately **30-60 seconds**. The Gunicorn timeout is set to 300 seconds to accommodate this. Subsequent restarts take less than 2 seconds (migration version guard detects schema is current).

Step 5 — Restart / Update Commands

```
# Restart all services docker-compose restart # Rebuild and restart after code changes docker-compose up --build -d # Stop all services docker-compose down # View live logs docker-compose logs -f
```

SECTION 14

Installation Guide — Manual Local Development Setup

Backend Setup

```
# 1. Navigate to backend directory cd backend # 2. Create Python virtual environment python -m venv venv # 3a. Activate (Linux/macOS) source venv/bin/activate # 3b. Activate (Windows PowerShell) venv\Scripts\Activate.ps1 # 4. Install all dependencies pip install -r requirements.txt # 5. Ensure backend/.env is set up (copy from .env.example) cp .env.example .env # Edit .env with your DATABASE_URL and other secrets # 6. Run the development server python run.py # Flask will start on http://127.0.0.1:5000 # Database tables created automatically on first run
```

Frontend Setup (Static File Serving)

```
# Option A: Python simple server (quickest for local dev) cd frontend python -m http.server 8080 # Visit http://localhost:8080 # Option B: Use VS Code Live Server extension # Right-click frontend/index.html → "Open with Live Server" # IMPORTANT: api.js must point to backend: # In frontend/assets/js/api.js, ensure BASE_URL = 'http://127.0.0.1:5000' # for local development
```

SECTION 15

Environment Variables — Complete Reference

All environment variables are loaded from `backend/.env` by `python-dotenv` in `app/config/settings.py`.

Variable	Required	Default	Description
`DATABASE_URL`	**Yes**	SQLite fallback	Full PostgreSQL connection string. Format: `postgresql://user:password@host:port/dbname` For Neon serverless, append `?sslmode=require`.
`JWT_SECRET_KEY`	**Yes**	`qcms_secret` (unsafe)	Secret used to sign JWT tokens. Must be a long random string in production. Minimum 32 characters.
`SECRET_KEY`	Recommended	`qcms_default_secret_key`	Flask session secret. Set to a different value from JWT_SECRET_KEY.
`RESEND_API_KEY`	**Yes** (for emails)	—	API key from resend.com for transactional email delivery (OTP, notifications, invites).
`RESEND_FROM_EMAIL`	**Yes** (for emails)	`onboarding@resend.dev`	The "From" email address. Must be a verified domain sender in Resend.
`SUPER_ADMIN_USERNAME`	**Yes**	—	Email of the platform-level super administrator account. Created automatically on first boot.
`SUPER_ADMIN_PASSWORD`	**Yes**	—	Password for the super admin account. Use a strong password.
`INTEGRATION_BASE_URL`	Recommended	`http://localhost:5000`	The public URL of the backend API. Used for Razorpay webhook callbacks and integration redirects.
`CORS_ORIGINS`	Recommended	`*` (unsafe)	Comma-separated list of allowed CORS origins. Set to your frontend domain in production, e.g., `https://app.yourdomain.com`.
`PORT`	No	`10000`	Port Gunicorn binds to. Render.com provides this automatically. Use `5000` for local Docker.
`GOOGLE_API_KEY`	No	—	Google API key for Google SSO / OAuth2 login integration (if enabled for an org).
`FLASK_ENV`	No	`production`	Set to `development` for debug mode with auto-reloader. Never use `development` in production.
`FLASK_APP`	No	—	Set to `run.py` if using `flask` CLI commands.

Critical Security Warning

NEVER commit `backend/.env` to git. The `.gitignore` is configured to exclude it, but always verify. If JWT_SECRET_KEY or DATABASE_URL is exposed, rotate them immediately and invalidate all existing sessions.

SECTION 16

Additional Configurations

16.1 — Gunicorn Production Settings

Configured in `backend/gunicorn.conf.py`:

Setting	Value	Reason
`workers`	1	Single worker prevents per-worker DB migration re-runs. Increase only after confirming migration guards work correctly.
`timeout`	300 seconds	Accommodates first-boot DB migration (100+ ALTER statements). Safe to reduce to 60s after first migration.
`preload_app`	True	App initialized once in master process. Workers inherit the migrated state — no duplicate migrations.
`worker_class`	sync	Synchronous worker is sufficient for QCMS request patterns.
`keepalive`	5	Persistent connections for Nginx reverse proxy.

16.2 — Nginx Reverse Proxy Configuration

Configured in `frontend/nginx.conf`:

- All `/api/*` requests are proxied to `http://backend:5000`
- All `/uploads/*` requests are proxied to backend for file serving
- Frontend SPA routing uses `try_files $uri $uri/ /index.html` for client-side navigation
- For production HTTPS: add SSL certificates via `ssl_certificate` directives or use a load balancer / Cloudflare for TLS termination

16.3 — Database Connection Pool

- Standard PostgreSQL: `pool_size=5`, `max_overflow=5`, `pool_recycle=1800`, `pool_pre_ping=True`
- Serverless (Vercel/Lambda): `NullPool` (no persistent connections — each request opens/closes its own connection)
- Neon Serverless: Requires `?sslmode=require` in DATABASE_URL

16.4 — File Upload Configuration

- Max file size: **16 MB** per upload (set in `MAX_CONTENT_LENGTH`)
- Upload folder: `backend/uploads/` (created automatically on boot)
- Serverless environments: uploads go to `/tmp/uploads` (ephemeral — consider S3/GCS for production serverless)
- Supported file types enforced per route (images, PDFs, Excel)

16.5 — Super Admin Account Bootstrap

- On first boot, `boot_utils.py` reads `SUPER_ADMIN_USERNAME` and `SUPER_ADMIN_PASSWORD` from env
- If no super admin exists in DB, one is created automatically with a platform-level organization
- Super Admin logs in at `/admin/` portal, not the regular organization login page

SECTION 17

Server Basic Requirements

17.1 — Minimum Requirements (Development / Small Org)

Component	Minimum	Recommended Production	Notes
CPU	2 vCPU	4 vCPU	RAG embedding computation is CPU-intensive during ingestion
RAM	1 GB	4 GB	all-MiniLM-L6-v2 model requires ~500MB RAM. Gunicorn + Nginx + PostgreSQL need additional headroom.
Disk	20 GB SSD	50 GB SSD	OS + Docker images (~2GB) + uploads + PDF reports + vector store
OS	Ubuntu 22.04 LTS	Ubuntu 22.04 LTS / Debian 12	Any Linux distro with Docker support works
Network	100 Mbps	1 Gbps	PDF report generation creates large binary responses
Open Ports	80 (HTTP), 443 (HTTPS)	80, 443 (+ firewall)	Port 5000 (backend) should NOT be publicly exposed — proxied via Nginx

17.2 — Software Requirements on Host

Software	Version	Required For
Docker	24+	Container runtime for both frontend and backend
Docker Compose	v2+	Multi-container orchestration
Python	3.11+	Only if running without Docker (manual setup)
PostgreSQL	15+	Only if hosting DB locally (Neon serverless is preferred)
Git	Any	Cloning the repository

17.3 — Cloud / Managed Service Options

Service	Provider	Purpose	Configuration Notes
Managed PostgreSQL	Neon (current), AWS RDS, Supabase	Primary DB	Add `?sslmode=require` to DATABASE_URL for Neon/cloud
Backend Hosting	Render, Railway, EC2, GCP VM	Flask API	Render provides PORT env var automatically. Gunicorn binds to it.
Frontend Hosting	Vercel, Netlify, S3+CloudFront	Static files	Update CORS_ORIGINS and INTEGRATION_BASE_URL to match
Transactional Email	Resend (current)	OTP, notifications	RESEND_API_KEY required. Verify sender domain in Resend dashboard.
Payments	Razorpay	Subscription billing	Configure Razorpay API keys in Super Admin → Integrations panel

SECTION 18

Key Integration Points

Integration	How It Works	Config Location
Razorpay Payments	Organization upgrades plan → Razorpay checkout → Webhook POST to `/api/subscriptions/razorpay/webhook` → subscription updated in DB	Super Admin → Integrations → Razorpay
Resend Email	Events (OTP, invite, alert) trigger `resend.Emails.send()` calls in `email_notification_routes.py`	`backend/.env`: RESEND_API_KEY
Google SSO	Optional per-org login option. OAuth2 flow via `GOOGLE_API_KEY`. Managed in org login_options JSONB.	`backend/.env`: GOOGLE_API_KEY + Admin → Login Options
RAG AI Engine	Sentence transformers + FAISS/PostgreSQL vector lookup via `POST /api/rag/chat`	Enterprise subscription required. Auto-activates.
External API Access	Enterprise orgs get API key for programmatic access. Integration endpoint at `/api/v1/integrations/`	Admin → API Access panel

SECTION 19

Troubleshooting & Common Issues

Symptom	Likely Cause	Fix
Backend container exits on startup	DATABASE_URL is wrong or DB unreachable	Check `docker-compose logs backend`. Verify DB URL, credentials, and network accessibility from container.
First boot takes 60+ seconds	Running 100+ ALTER TABLE migrations against remote DB	Normal. Wait for "Schema migrations complete" in logs. Subsequent boots are fast.
401 on all API calls after code deploy	JWT_SECRET_KEY changed between deployments	All existing tokens are now invalid. Users must log in again. This is correct behavior.
CORS errors in browser	CORS_ORIGINS not set to match frontend domain	Set `CORS_ORIGINS=https://yourdomain.com` in `backend/.env` and restart.
File upload fails with 413	Nginx rejecting file larger than default limit	Add `client_max_body_size 20M;` to nginx.conf server block.
Emails not sending	RESEND_API_KEY invalid or from-email domain not verified	Verify key in Resend dashboard. Check RESEND_FROM_EMAIL is a verified sender.
Super Admin login fails	SUPER_ADMIN_PASSWORD not set or account not bootstrapped	Check `boot_utils.py` ran. Verify SUPER_ADMIN_USERNAME and SUPER_ADMIN_PASSWORD in .env on first boot.
RAG AI returns no results	No projects have been ingested into knowledge_repository yet	At least one project must be Closed and Approved for ingestion to trigger.

SECTION 20

KT Handoff Checklist

Access & Credentials to Hand Over

- GitHub repository access
- Neon / PostgreSQL DB connection string

Knowledge Items to Verify

- Understand the 8-stage workflow & approval gates
- Know how to add a new organization (Super Admin)

- Resend API key + verified sender domain
- Razorpay API key + webhook secret
- Super Admin login credentials
- Docker registry (if using private images)
- Render / hosting platform access
- Google API Console project (for SSO)

- Know how the DB auto-migration version guard works
- Know how to read audit logs for compliance events
- Know how to force-terminate a user session
- Know how to run a manual DB migration if needed
- Understand the subscription tier feature gates
- Know how to trigger RAG ingestion for a closed project

Quick Health Check Command

After any deployment, run: `curl https://yourdomain.com/api/auth/health`. If it returns `{"status": "ok"}` the backend is healthy. Then log in as Super Admin and verify the Organizations list loads correctly.